

Introduction to Algorithm

기초 알고리즘

Divide and Conquer 개요

재귀 호출



Q. 1부터 n까지 더하는 작업을 코드로 작성하면?

>

단순한 반복 작업을 코드로 구현하는 미션이 주어졌을 때,
가장 먼저 떠오르는 문법은?



Q. 1부터 n까지 더하는 작업을 코드로 작성하면?

```
result = 0
for i in range(1, n + 1):
    result += i
```

for 문으로 해결할 수 있고,



Q. 1부터 n까지 더하는 작업을 코드로 작성하면?

```
result = 0  
i = 1  
while i <= n:  
    result += i  
    i += 1
```

while 문으로 해결할 수도 있음



Q. 1부터 n 까지 더하는 작업을 코드로 작성하면?

>

하지만, 이 밖에도 정말 유용하고 편리한 방법이 하나 더 있는데,



Q. 1부터 n까지 더하는 작업을 코드로 작성하면?

```
def adder1(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder1(n - 1)
```

바로 재귀 호출



Q. 1부터 n까지 더하는 작업을 코드로 작성하면?

```
def adder1(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder1(n - 1)
```

adder1() 함수가 수행된다면

마지막 줄에서 반환할 때 adder1() 함수를 호출



Q. 1부터 n까지 더하는 작업을 코드로 작성하면?

```
def adder1(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder1(n - 1)
```

`adder1()` 함수가 호출되었으니 다시 첫 번째 줄부터 수행되고
마지막 줄에서 반환할 때 `adder1()` 함수를 또 호출



Q. 1부터 n까지 더하는 작업을 코드로 작성하면?

```
def adder1(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder1(n - 1)
```

이렇게 자기 자신을 호출하는 것을 재귀라고 함



재귀 호출은 함수 내부에서 자기 자신을 호출하는 방법

재귀 호출은 함수에서만 사용되기 때문에

재귀 호출 함수 혹은 **재귀 함수**로 불림



다시 재

再歸

돌아갈 귀

재귀란 “본디의 곳으로 다시 돌아온다”라는 뜻을 지닌 한자

재귀 함수는 한번 호출되면

“본디 함수가 정의된 곳으로 다시 돌아오기” 때문



재귀 호출 동작 방식

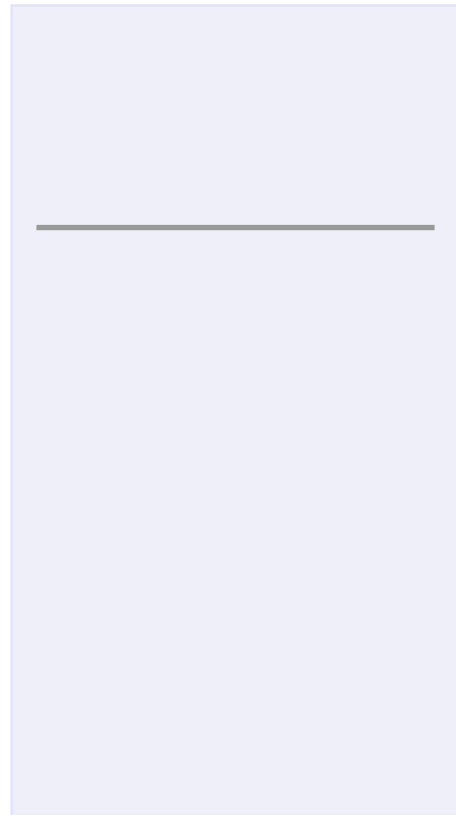
```
def adder1(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder1(n - 1)  
  
def main():  
    x = 0  
    x = adder(2)  
    print(x)    # 3
```

위와 같이 작성된 `main()` 함수를 실행하면,
1부터 2까지 더한 3이 출력됨. 과정을 따라가보자



재귀 호출 동작 방식

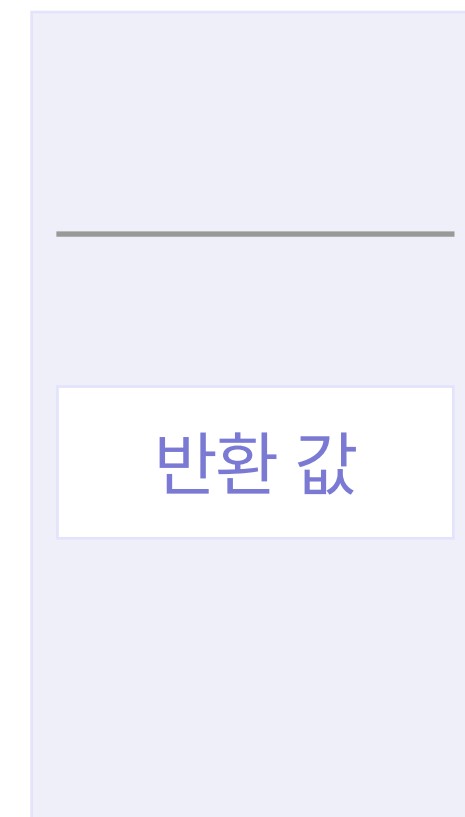
스택(Stack)



함수 호출



함수 실행 완료



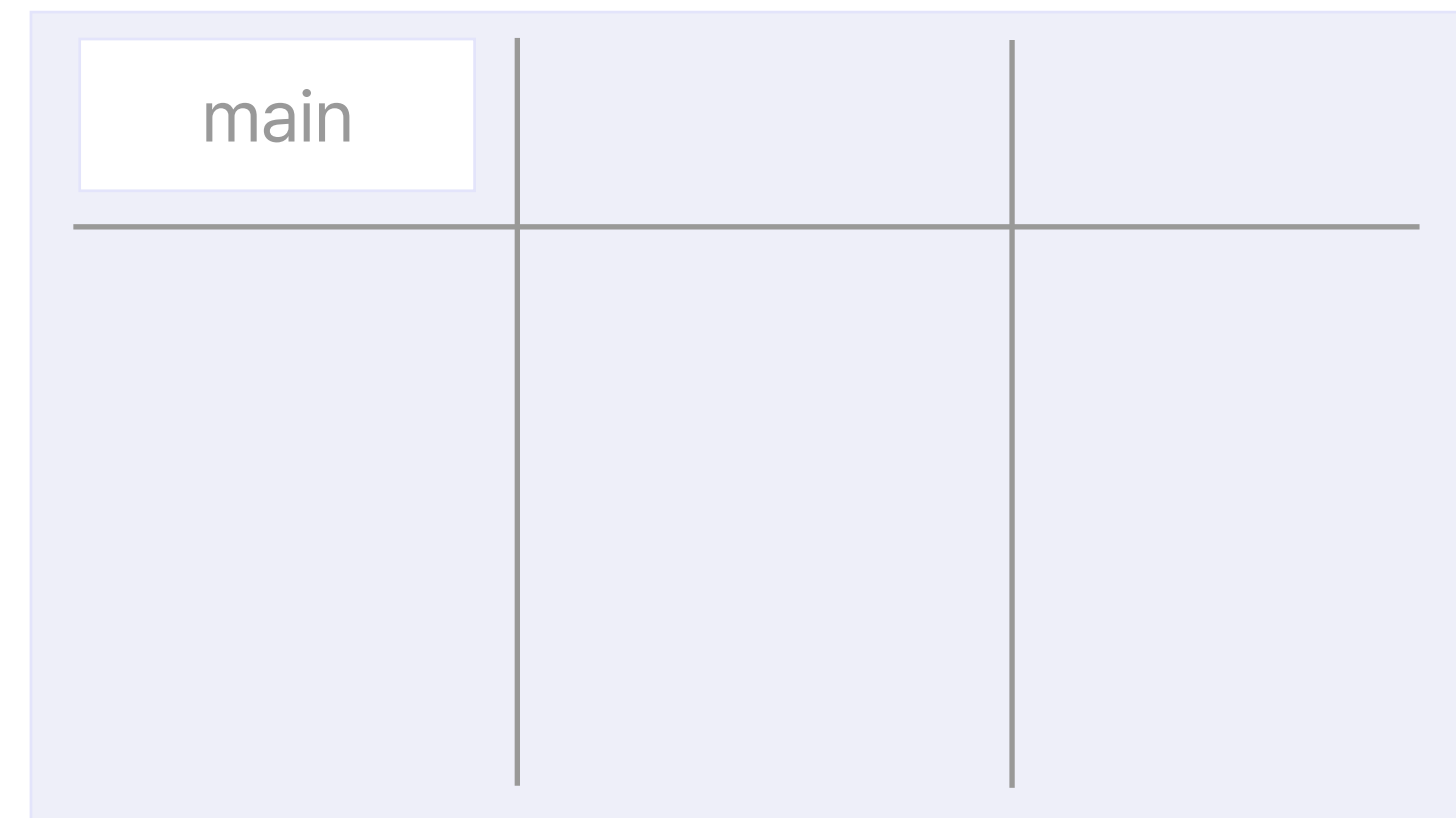


재귀 호출 동작 방식

수행 코드

```
main()
```

스택



컴퓨터는 `main()` 함수를 호출

컴퓨터는 이제 `main()` 함수에 담긴 코드를 한 줄씩 읽기 시작

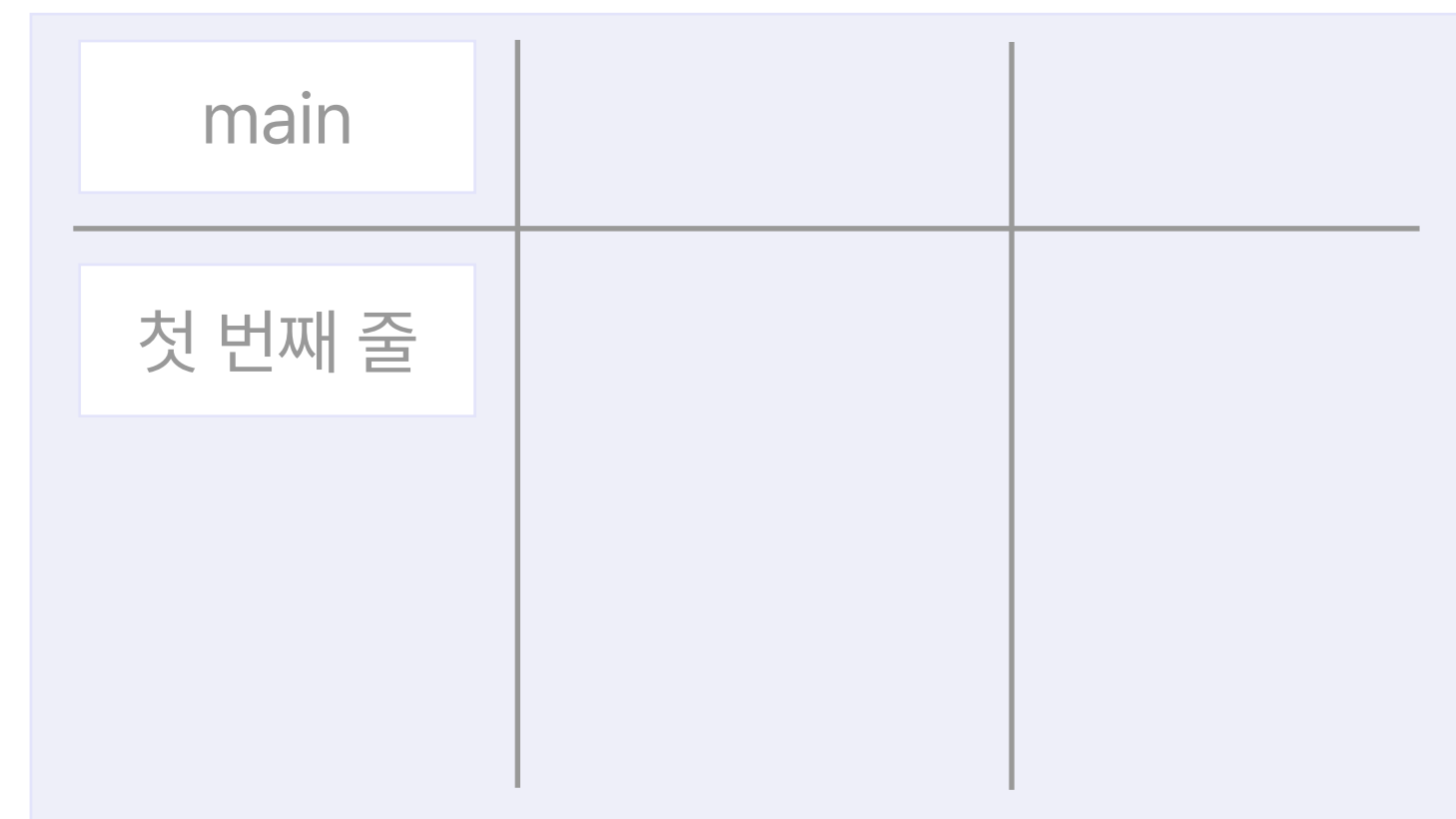


재귀 호출 동작 방식

수행 코드

```
def main():  
    x = 0  
    ...
```

스택



컴퓨터는 `main()` 함수를 호출

컴퓨터는 이제 `main()` 함수에 담긴 코드를 한 줄씩 읽기 시작

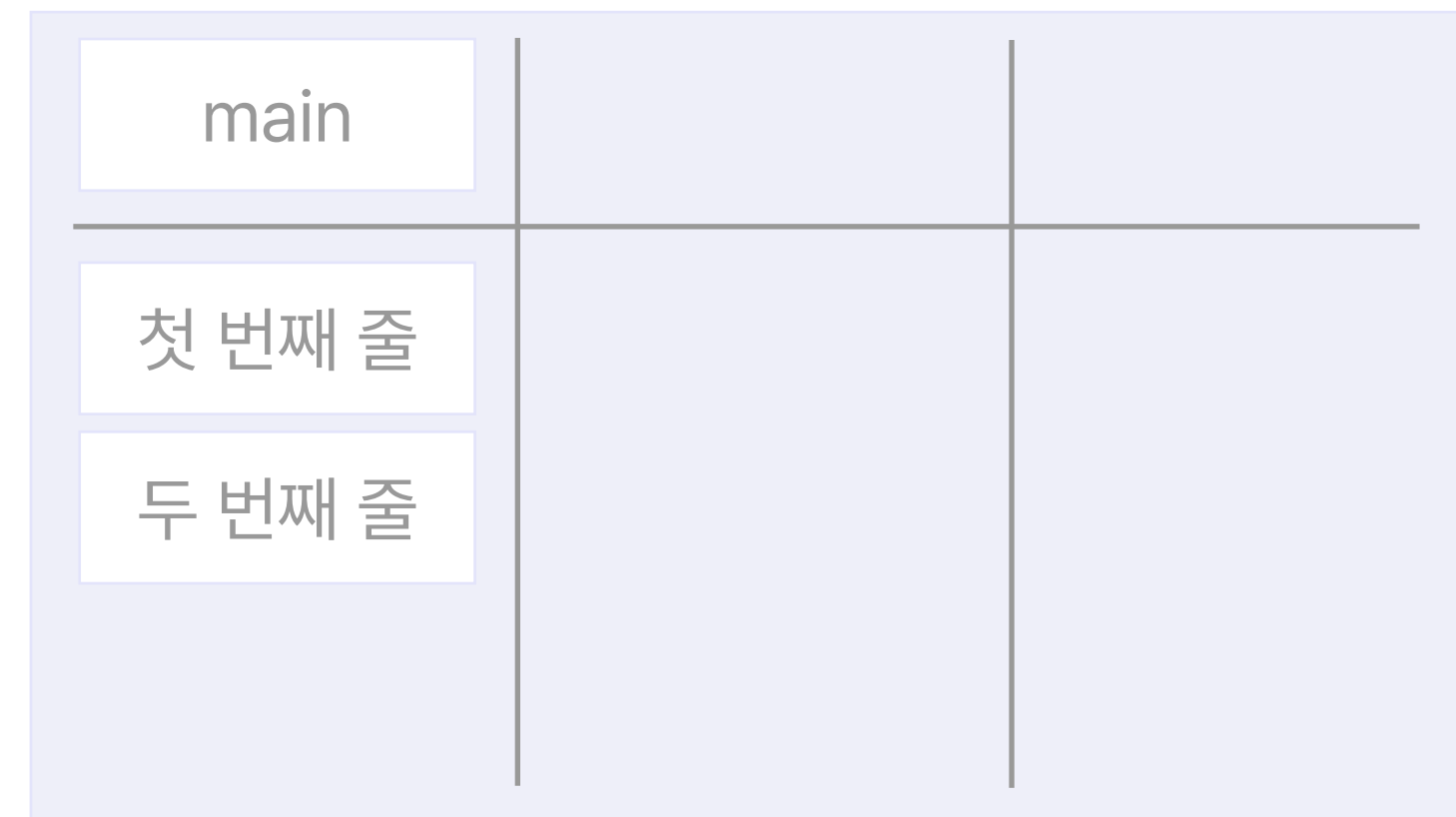


재귀 호출 동작 방식

수행 코드

```
def main():  
    ...  
    x = adder(2)  
    ...
```

스택



main() 함수의 두 번째 코드를 읽으려고 했는데,
adder(2) 함수를 발견



재귀 호출 동작 방식

수행 코드

```
def main():  
    ...  
    x = adder(2)  
    ...
```

스택



컴퓨터는 adder(2) 함수를 실행해 결과값을 먼저 반환받아야 함
그래야 x에 반환 값을 저장할 수 있음

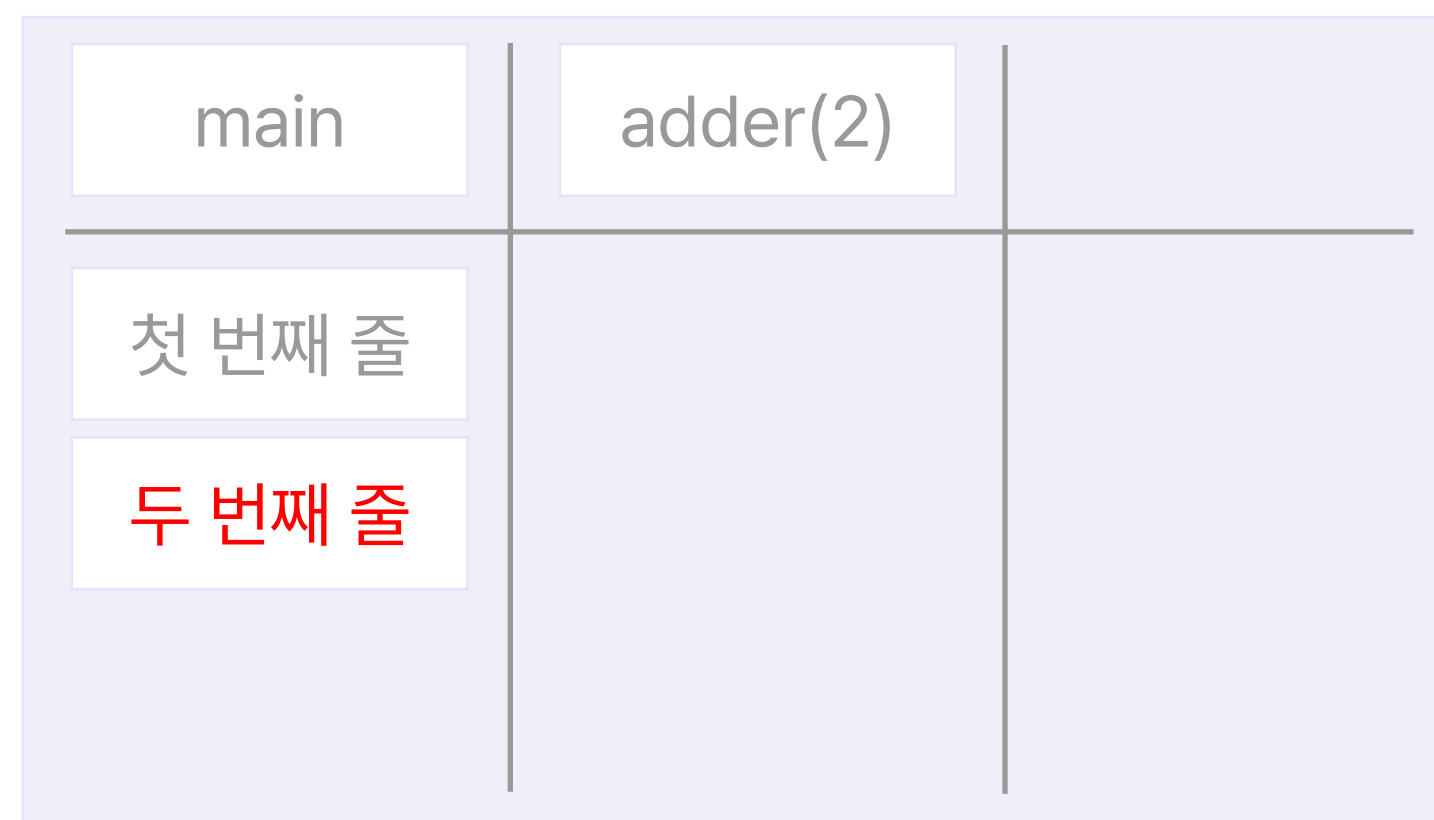


재귀 호출 동작 방식

수행 코드

```
def main():  
    ...  
  
    x = adder(2)  
  
    ...
```

스택



컴퓨터는 main() 함수 두 번째 코드 실행을 완료하기 위해
adder(2) 함수를 위한 공간을 마련



재귀 호출 동작 방식

수행 코드

```
def adder(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder(n - 1)
```

스택



컴퓨터는 이제 adder(2) 함수를 수행

adder(2) 함수를 보니, 매개변수 n의 값으로 들어온 숫자는 2

들어온 매개변수에 따라 조건문 분기를 수행하면 2 + adder(1) 반환



재귀 호출 동작 방식

수행 코드

```
def adder(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder(n - 1)
```

스택



마찬가지로 컴퓨터는 `adder(1)` 함수를 실행해
결괏값을 먼저 반환받아야 함



재귀 호출 동작 방식

수행 코드

```
def adder(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder(n - 1)
```

스택



adder(1) 함수를 보니, n이 1
결과적으로 adder(1) 함수는 1을 반환



재귀 호출 동작 방식

수행 코드

```
def adder(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder(n - 1)
```

스택



컴퓨터는 반환 값을 adder(2) 함수로 반환



재귀 호출 동작 방식

수행 코드

```
def adder(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder(n - 1)
```

스택



adder(2) 함수의 네 번째 코드는 이제 온전하게 계산됨

$2 + \text{adder}(1) = 2 + 1 = 3$, 즉 3을 반환



재귀 호출 동작 방식

수행 코드

```
def adder(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder(n - 1)
```

스택



adder(2) 함수의 네 번째 코드는 이제 온전하게 계산됨

$2 + \text{adder}(1) = 2 + 1 = 3$, 즉 3을 반환



재귀 호출 동작 방식

수행 코드

```
def main():  
    ...  
    x = adder(2)  
    ...
```

스택



다시 main() 함수로 넘어와
x에는 adder(2) 함수의 반환 값, 즉 3을 저장

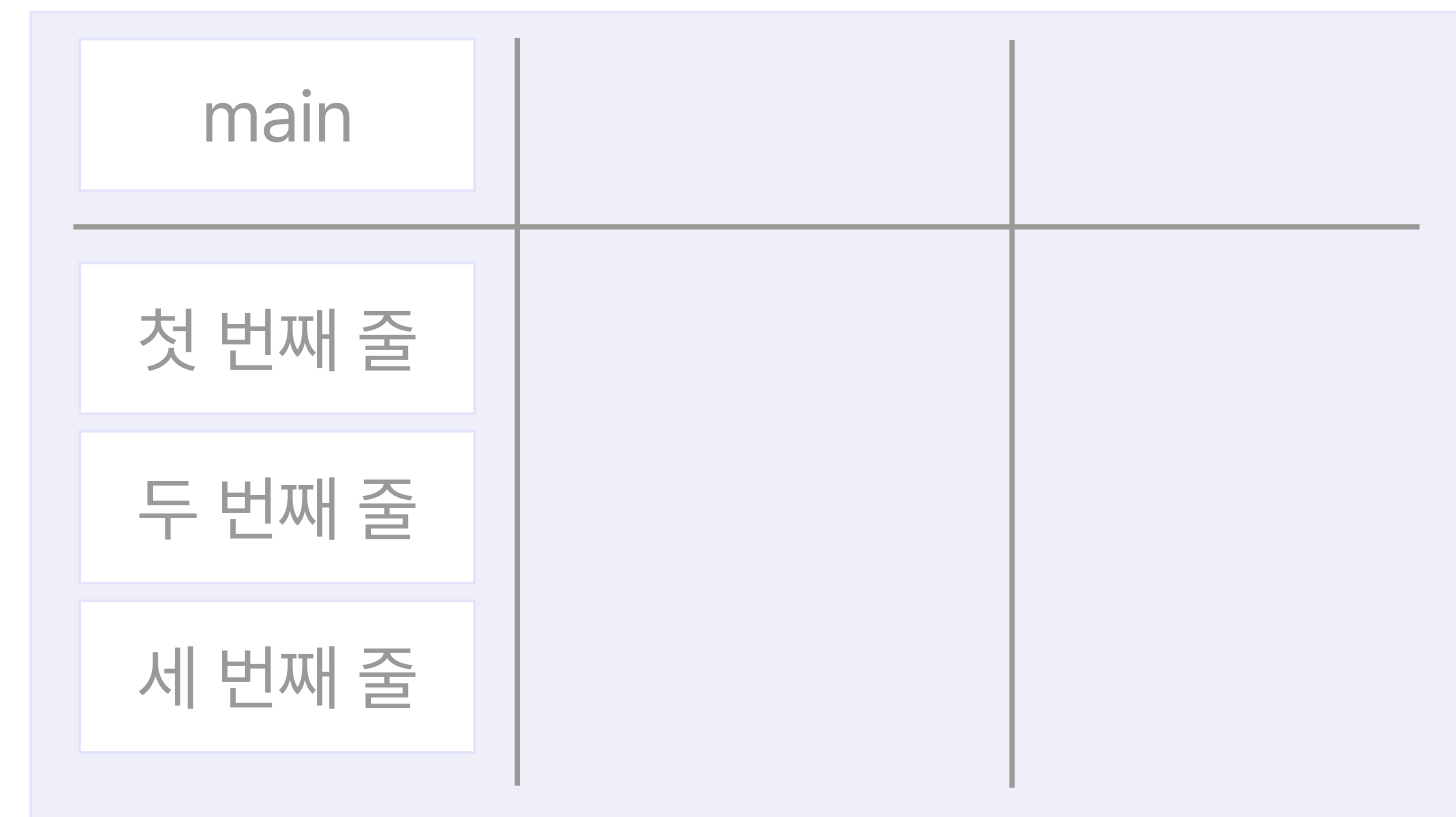


재귀 호출 동작 방식

수행 코드

```
def main():  
    ...  
    ...  
    print(x)
```

스택



이렇게 두 번째 줄까지 완료되고,
main() 함수의 세 번째 줄을 읽기

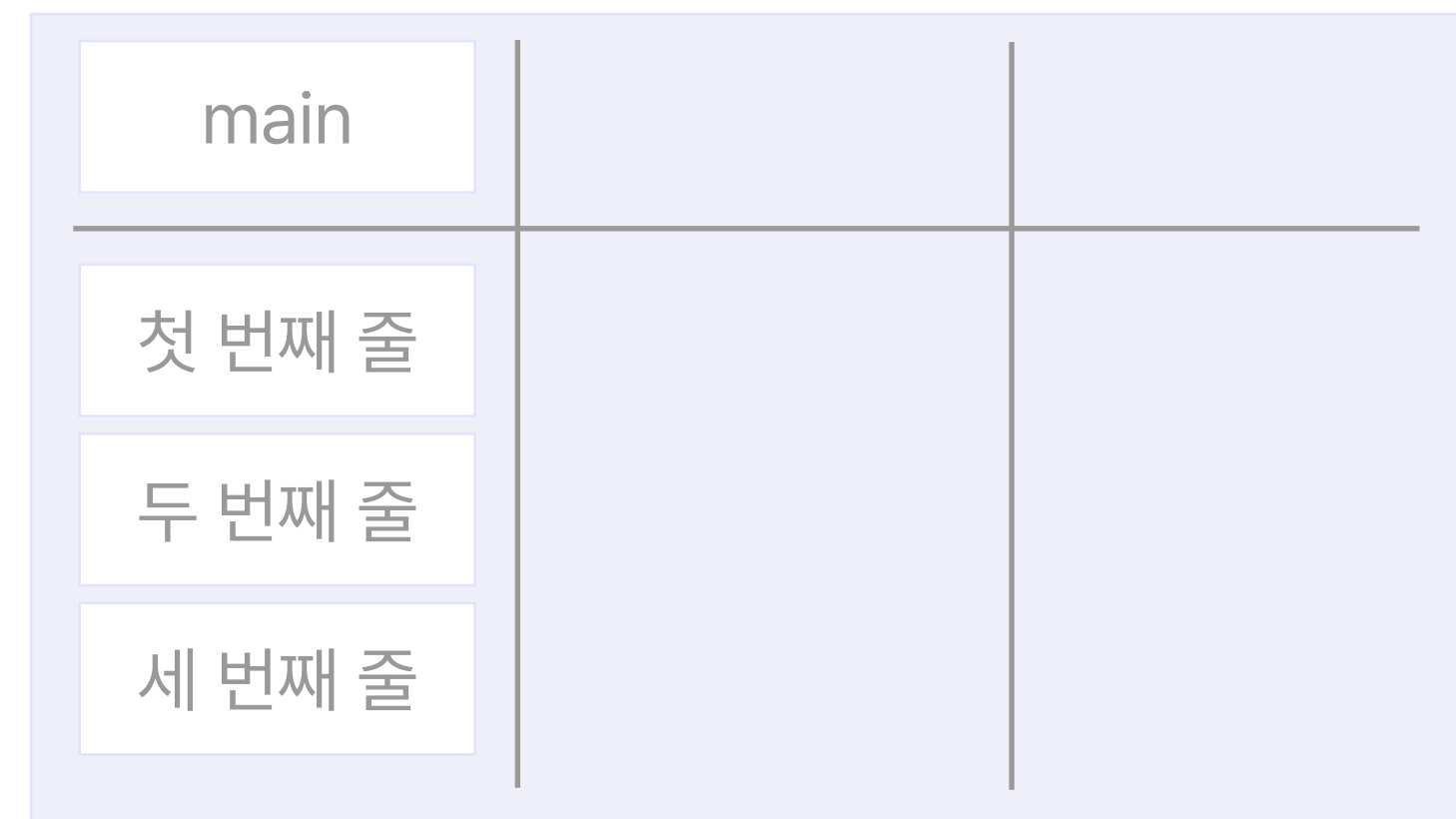


재귀 호출 동작 방식

수행 코드

```
def main():  
    ...  
    ...  
    print(x) # 출력값 3
```

스택



출력까지 완료하고 나면,
main() 함수도 실행 완료



재귀함수를 사용하는 이유



```
def fibonacci(n):  
    if n <= 2:  
        return 1  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

변수의 사용을 줄여, 프로그램을 더 간결하고 이해하기 쉽게 만들 수 있음

재귀 함수는 복잡한 문제를 간단한 기본 단계(base case)로 나누고,

이러한 기본 단계에서 해결하는 방식을 통해 문제를 해결할 수 있게 함



재귀함수 vs. 반복문

구분	반복문	재귀
기본	명령을 반복적으로 실행	함수 자체를 호출
구조	초기화, 조건, 루프 내 명령문 실행과 제어 변수 업데이트 포함	종료 조건만 지정(조건이 추가될 수도 있음)
종료	설정한 조건에 도달 할 때까지 반복 실행	함수 호출 본문에 조건부가 포함, 재귀를 호출하지 않고 함수를 강제 반환
조건	제어 조건이 참이라면 무한 반복 발생	조건에 수렴하지 않을 경우 무한 재귀 발생
무한 반복	무한 루프는 CPU 사이클을 반복적으로 사용	무한 재귀는 스택 오버플로우 발생
스택 메모리	스택 메모리를 사용하지 않음	함수가 호출 될 때마다 새 로컬 변수와 매개 변수 집합, 함수 호출 위치를 저장하는데 사용
속도	빠른 실행	느린 실행
가독성	코드 길이가 길어지고 변수가 많아져 가독성이 떨어짐	코드 길이와 변수가 적어 가독성이 높아짐



$$\text{Factorial}(n) = n \times (n-1) \times \dots \times 2 \times 1$$

팩토리얼(Factorial)이란 1부터 n까지의 곱

재귀 호출로 작성해보면?



```
def Factorial(n):  
    pass
```

처음에는 팩토리얼 함수를 정의



```
def Factorial(n):  
    return n * Factorial(n - 1)
```

그 다음, 처리해야 하는 핵심 문제 상황을 코드로 작성



```
def Factorial(n):  
    if n == 1:    # 탈출 조건  
        return 1  
    else:  
        return n * Factorial(n - 1)
```

재귀 호출은 **탈출 조건**이 없으면 무한하게 동작하기 때문에
반드시 **탈출 조건**을 작성해줘야 함



재귀 함수를 작성할 때의 주의점

- 무한 루프에 빠지지 않도록 종료 조건을 잘 설정
 - 종료 조건을 **기저 사례** (Base Case) / **기저 조건** (Base Condition) 이라고도 함
- 함수의 파라미터 및 인자 설정에 유의 : 필요한 변수, 리스트가 있다면 과감하게 재귀함수 인풋에 같이 넣어주자! (Call by Reference!)



알고리즘 입문자들이 어려워하기 쉬운 부분이 재귀
재귀는 이후에 배울 알고리즘의 밑바탕이 되니 꼭 숙지!



재귀함수를 디자인하기 위해서는 다음 세 가지를 명심하자.

1. 함수의 **정의**를 명확히 한다.
2. **기저 조건(Base condition)**에서 함수가 제대로 동작하게 작성한다.
3. 그 후, 함수가 (작은 input에 대하여) **제대로 동작**한다고 가정하고 함수를 완성한다.



[예제] 가장 가까운 값 찾기

정렬된 n 개의 숫자 중 정수 m 과 가장 가까운 값을 찾아라
단, $1 \leq n \leq 100,000$

입력 예시

```
1 4 6 7 10 14 16
8
```

출력 예시

```
7
```




[예제] 가장 가까운 값 찾기

재귀함수를 디자인하기 위해서는 다음 세 가지를 명심하자.

1. 함수의 **정의**를 명확히 한다.

```
getNearest(data, m)
```

data에서

m보다 작거나 같은 값 중 최댓값,
m보다 큰 값 중 최솟값을 반환하는 함수



[예제] 가장 가까운 값 찾기

재귀함수를 디자인하기 위해서는 다음 세 가지를 명심하자

2. **기저 조건(Base condition)**에서 함수가 제대로 동작하게 작성한다.

$$n = 1$$



$$(-\infty, 16)$$



[예제] 가장 가까운 값 찾기

재귀함수를 디자인하기 위해서는 다음 세 가지를 명심하자

2. **기저 조건(Base condition)**에서 함수가 제대로 동작하게 작성한다.

$$n = 2$$

10	14	8
----	----	---

$$(-\infty, 10)$$



[예제] 가장 가까운 값 찾기

재귀함수를 디자인하기 위해서는 다음 세 가지를 명심하자

2. **기저 조건(Base condition)**에서 함수가 제대로 동작하게 작성한다.

$$n = 2$$

2	6	8
---	---	---

$$(6, \infty)$$



[예제] 가장 가까운 값 찾기

재귀함수를 디자인하기 위해서는 다음 세 가지를 명심하자

2. **기저 조건(Base condition)**에서 함수가 제대로 동작하게 작성한다.

$$n = 2$$

7	10	8
---	----	---

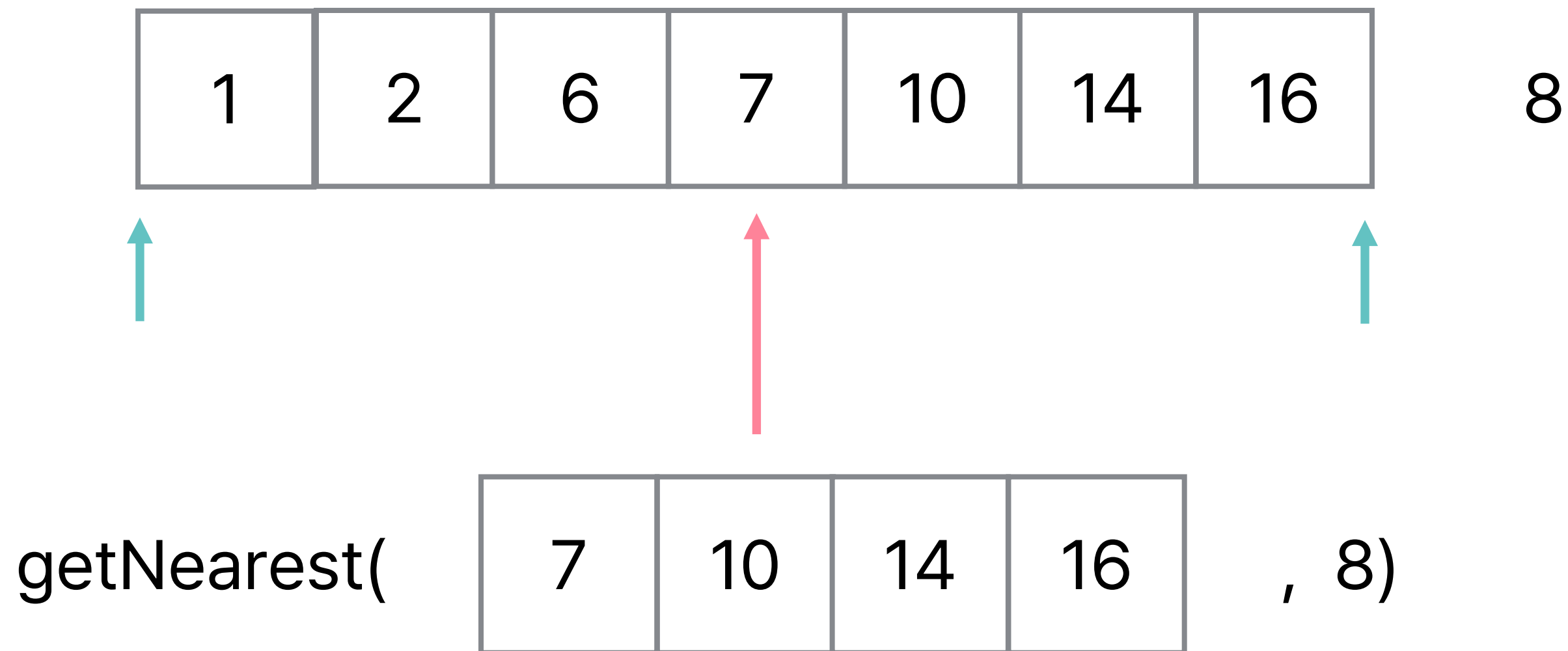
(7, 10)



[예제] 가장 가까운 값 찾기

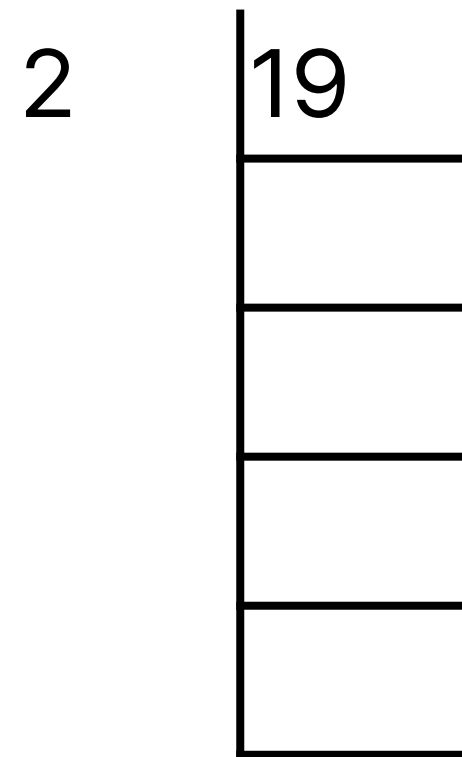
재귀함수를 디자인하기 위해서는 다음 세 가지를 명심하자

3. 그 후, 함수가 (작은 input에 대하여) **제대로 동작한다고 가정하고** 함수를 완성한다.





[예제] 이진수 변환



19를 이진수로 표현하기 위해서
더 나눌 수 없을 때까지 2로 나뉘어야 함



[예제] 이진수 변환

$$\begin{array}{r|l} 2 & 19 \\ \hline & 9 \quad 1 \\ \hline & \\ \hline & \\ \hline & \\ \hline & \end{array}$$

19를 2로 나누고 몫과 나머지를 구함



[예제] 이진수 변환

$$\begin{array}{r|l} 2 & 19 \\ \hline 2 & 9 \quad 1 \\ \hline & 4 \quad 1 \\ \hline & \\ \hline & \end{array}$$

다시 9를 2로 나누고 몫과 나머지를 구함



[예제] 이진수 변환

$$\begin{array}{r|l} 2 & 19 \\ \hline 2 & 9 \quad 1 \\ \hline 2 & 4 \quad 1 \\ \hline & 2 \quad 0 \\ \hline & \end{array}$$

다시 4를 2로 나누고 몫과 나머지를 구함



[예제] 이진수 변환

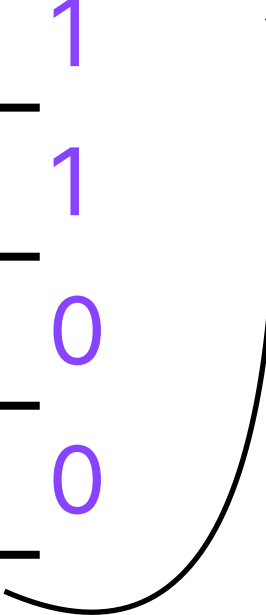
$$\begin{array}{r|l} 2 & 19 \\ \hline 2 & 9 \quad 1 \\ \hline 2 & 4 \quad 1 \\ \hline 2 & 2 \quad 0 \\ \hline & 1 \quad 0 \end{array}$$

마지막으로 2를 2로 나누고 몫과 나머지를 구함



[예제] 이진수 변환

2	19	
2	9	1
2	4	1
2	2	0
	1	0



더 나눌 수 없을 때까지 나눴으니 알고리즘은 종료
이진수는 바로 마지막으로 남은 몫, 그리고 모든 나머지 값들



[예제] 이진수 변환

$$19 = 1 * 16 + 0 * 8 + 0 * 4 + 1 * 2 + 1 * 1$$

2	19	
2	9	1
2	4	1
2	2	0
	1	0

더 나눌 수 없을 때까지 나눴으니 알고리즘은 종료
이진수는 바로 마지막으로 남은 몫, 그리고 모든 나머지 값들



[예제] 이진수 변환

```
def convertBinary(n):  
    if n == 0:    # 탈출 조건  
        return "0"  
    if n == 1:    # 탈출 조건  
        return "1"  
    else:  
        return convertBinary(n // 2) + str(n % 2)
```



[예제] 거듭제곱 구하기

$$m^n = m \times m \times \dots \times m$$

getPower(m, n) : m^n 을 반환하는 함수

점화식

$$\text{getPower}(m, n) = m \times \text{getPower}(m, n-1)$$

$$\text{getPower}(m, 0) = 1$$

기저조건
(Base condition)



[예제] 거듭제곱 구하기

$$m^n = m \times m \times \dots \times m$$

getPower(m, n) : m^n 을 반환하는 함수

```
def getPower(m, n) :  
    if n == 0 :  
        return 1  
    else :  
        return m * getPower(m, n-1)
```

$O(n)$



[예제] 거듭제곱 구하기

$$m^n = m \times m \times \dots \times m$$

getPower(m, n) : m^n 을 반환하는 함수

$$m^n = (m^{(n/2)})^2 \quad n\text{이 짝수일 경우}$$

$$(m^{n-1}) \times m \quad n\text{이 홀수일 경우}$$



[예제] 거듭제곱 구하기

$$m^n = m \times m \times \dots \times m$$

getPower(m, n) : m^n 을 반환하는 함수

$$\text{getPower}(m, n) = (\text{getPower}(m, n//2))^2 \quad n\text{이 짝수}$$

$$\text{getPower}(m, n) = m \times \text{getPower}(m, n-1) \quad n\text{이 홀수}$$



[예제] 거듭제곱 구하기

$$m^n = m \times m \times \dots \times m$$

getPower(m, n) : m^n 을 반환하는 함수

```
def getPower(m, n) :  
    if n == 0 :  
        return 1  
    elif n % 2 == 0 :  
        temp = getPower(m, n//2)  
        return temp * temp  
    else :  
        return getPower(m, n-1) * m
```

$O(\log n)$

분할 정보



분할 정복 (Divide and Conquer)

- **Divide** → 큰 문제를 작은 문제로 분할
 - 기저 사례(base case)를 잘 설정하여 일정 기준 이상 분할되지 않도록 해야 함
- **Conquer** → 작은 문제의 답을 모아, 큰 문제의 답을 구함
- 분할 정복은 재귀로 구현하는 것이 일반적

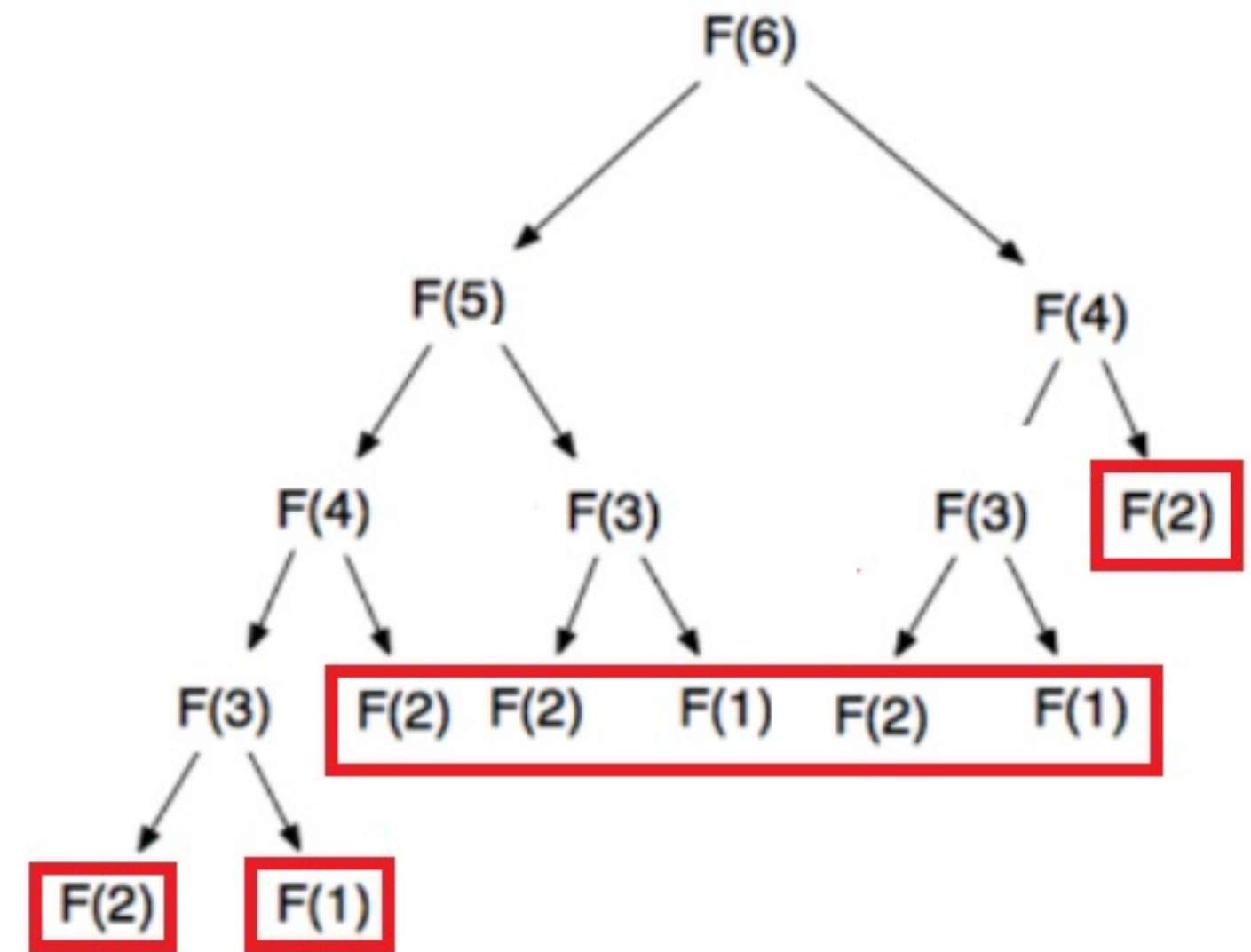
6 5 3 1 8 7 2 4



분할 정복 (Divide and Conquer)

예시 - 피보나치 수열

```
def fibo(N):  
    if N <= 2:  
        return 1  
    return fibo(N - 1) + fibo(N - 2)
```



N 이 2이하일 때, 1 반환 → 기저 사례(base case)

$F(1)$ 값을 3번, $F(2)$ 값을 5번 계산하여, $F(6)$ 을 구함

→ 큰 문제를 작은 문제로 분할 + 작은 문제들의 답으로 큰 문제의 답을 구함



분할 정복 (Divide and Conquer)

예시 - Z

0	1
2	3

0	1	4	5
2	3	6	7
8	9	12	13
10	11	14	15

0	1	4	5	16	17	20	21
2	3	6	7	18	19	22	23
8	9	12	13	24	25	28	29
10	11	14	15	26	27	30	31
32	33	36	37	48	49	52	53
34	35	38	39	50	51	54	55
40	42	44	45	56	57	60	61
41	43	46	47	58	59	62	63

크기가 $2^n \times 2^n$ 인 2차원 배열이 존재
 n 의 값이 주어질 때, r 행 c 열을 몇 번째로 방문하는지 알고 싶다!



분할 정복 (Divide and Conquer)

예시 - Z

```
import sys
result = 0
def sol(n, x, y):
    global result
    if x == r and y == c:
        print(int(result))
        return

    if not (x <= r < x + n and y <= c < y + n):
        result += n * n
        return

    sol(n / 2, x, y)
    sol(n / 2, x, y + n / 2)
    sol(n / 2, x + n / 2, y)
    sol(n / 2, x + n / 2, y + n / 2)

n, r, c = map(int, sys.stdin.readline().split(' '))
sol(1 << n, 0, 0)
```

전역 변수 result를 0으로 초기화

함수 $sol(n, x, y)$ 를 정의:

만약 x 가 r 이고 y 가 c 인 경우:

result를 정수로 변환하여 출력 및 return

만약 (x, y) 에서 시작해서 크기가 n 인 사각형이 (r, c) 를 포함하지 않는 경우:

result에 $n * n$ 을 더하고 return

4개의 재귀 호출을 사용하여 사분면으로 나눔:

$sol(\frac{n}{2}, x, y)$ 를 호출 (2사분면)

$sol(\frac{n}{2}, x, y + \frac{n}{2})$ 를 호출 (1사분면)

$sol(\frac{n}{2}, x + \frac{n}{2}, y)$ 를 호출 (3사분면)

$sol(\frac{n}{2}, x + \frac{n}{2}, y + \frac{n}{2})$ 를 호출 (4사분면)

사용자로부터 n, r, c 를 입력 받음

$sol(2^n, 0, 0)$ 호출



분할 정복 (Divide and Conquer)



분할 정복법이란, 주어진 큰 문제를 작은 단위로 분할해서 작은 문제를 하나씩 해결함으로써 큰 문제를 해결하는 프로그래밍 기법



[예제] 연속부분 최대합

연속된 부분을 선택하였을 때, 그 최대 합을 출력
단, $1 \leq n \leq 100,000$

입력 예시

1 2 -4 5 3 -2 9 10

출력 예시

15



분할 정복법을 적용해본다면?

우선 절반으로 나누어 각각을 구해보자

2	1	-2	5	-10	3		2	5	-3	7	9	-10
---	---	----	---	-----	---	--	---	---	----	---	---	-----



분할 정복법을 적용해본다면?

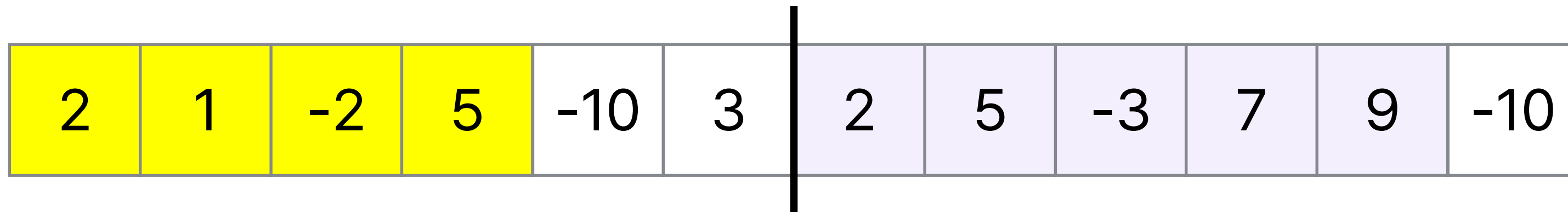
우선 절반으로 나누어 각각을 구해보자

2	1	-2	5	-10	3		2	5	-3	7	9	-10
---	---	----	---	-----	---	--	---	---	----	---	---	-----



분할 정복법을 적용해본다면?

우선 절반으로 나누어 각각을 구해보자

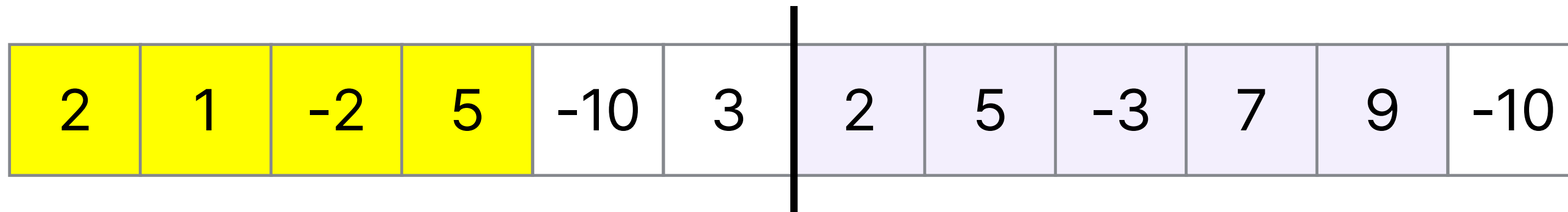


고려하지 않은 경우는 ?
자른 지점을 포함하는 연속 부분!



분할 정복법을 적용해본다면?

우선 절반으로 나누어 각각을 구해보자



자른 지점을 포함하는 연속 부분의 최대 합은 어떻게 구할까?

Idea : 왼쪽과 오른쪽을 독립적으로 생각하자



[예제] 연속부분 최대합

2	1	-2	5	-10	3		2	5	-3	7	9	-10
---	---	----	---	-----	---	--	---	---	----	---	---	-----

					3	3
				-10	3	-7
			5	-10	3	-2
		-2	5	-10	3	-4
	1	-2	5	-10	3	-3
2	1	-2	5	-10	3	-1



[예제] 연속부분 최대합

2	1	-2	5	-10	3	2	5	-3	7	9	-10
---	---	----	---	-----	---	---	---	----	---	---	-----

					3	3	2	2						
					-10	3	-7	7	2	5				
				5	-10	3	-2	4	2	5	-3			
		-2	5	-10	3	-4	11	2	5	-3	7			
	1	-2	5	-10	3	-3	20	2	5	-3	7	9		
2	1	-2	5	-10	3	-1	10	2	5	-3	7	9	-10	



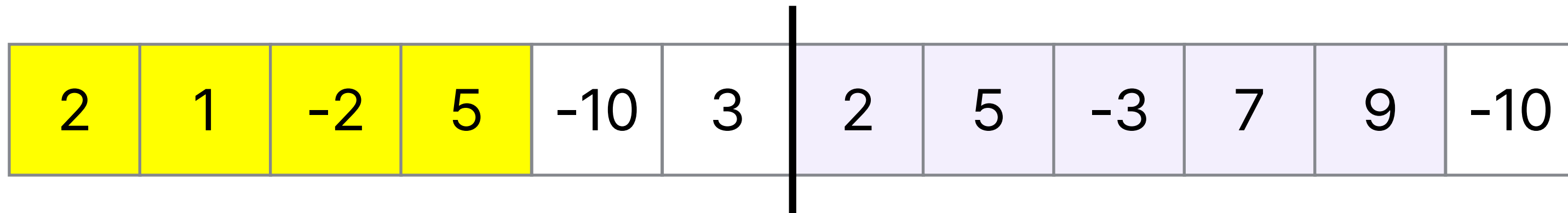
					3	3	2	2						
				-10	3	-7	7	2	5					
			5	-10	3	-2	4	2	5	-3				
		-2	5	-10	3	-4	11	2	5	-3	7			
	1	-2	5	-10	3	-3	20	2	5	-3	7	9		
2	1	-2	5	-10	3	-1	10	2	5	-3	7	9	-10	



[예제] 연속부분 최대합

모든 경우를 고려했음

1. 왼쪽만 포함하는 경우,
2. 오른쪽만 포함하는 경우,
3. 자른 자리를 포함하는 경우

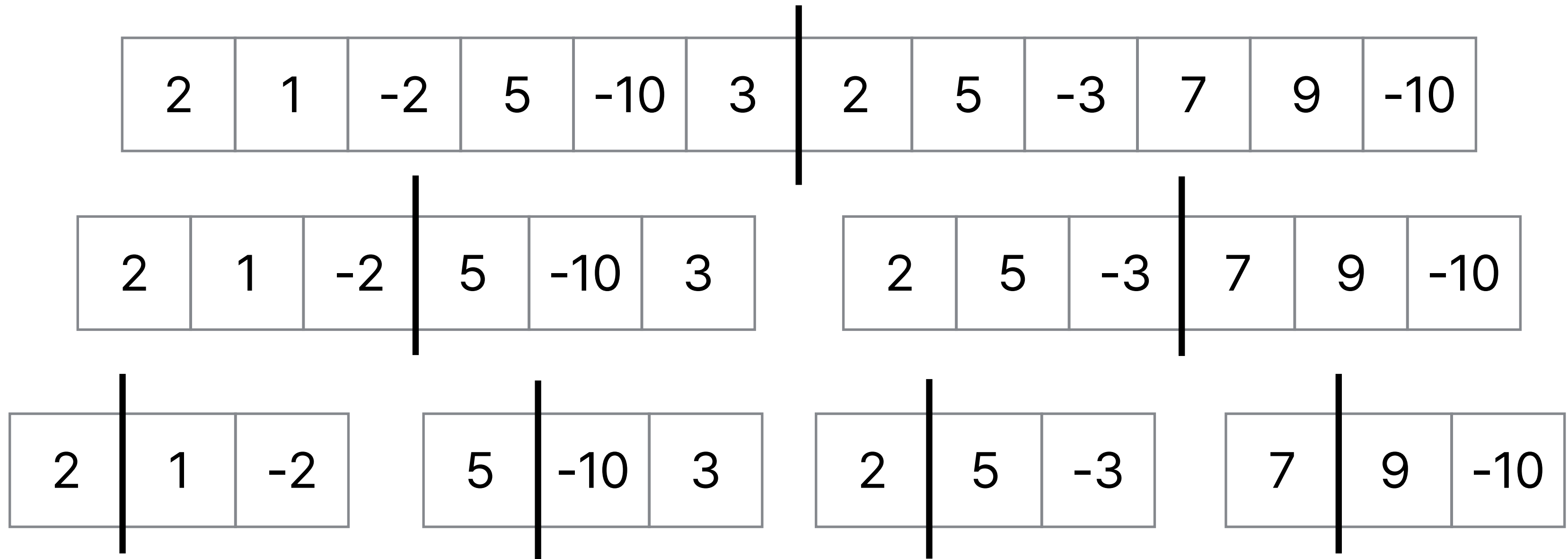


최댓값!





[예제] 연속부분 최대합



시간복잡도 : (왼쪽/오른쪽으로 나누기) x (연속된 경우 최대값 계산)

$O(N \log N)$

$O(\log N)$

$O(N)$



[예제] 색종이

입력 예시

```
4
1 0 1 1
0 1 1 1
0 0 1 1
0 1 1 1
```

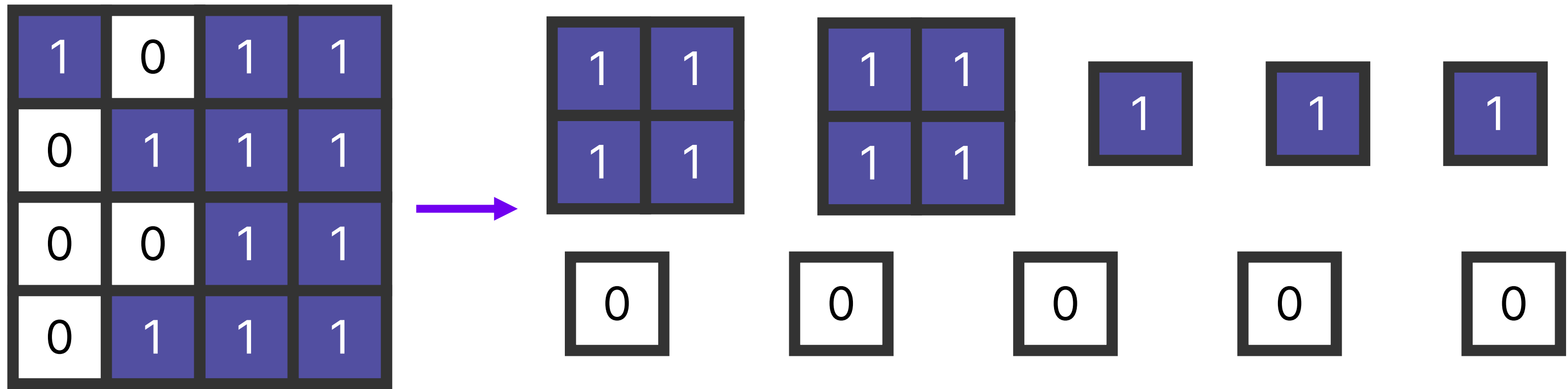
출력 예시

```
5
5
```

첫 줄에 색종이의 **한 변의 길이**가 들어오고,
이어서 색종이의 특정 칸의 **색상의 정보**가 들어옴



[예제] 색종이



다음과 같은 정사각형 모양의 색종이가 있을 때,
이 색종이를 잘라서 한 가지 색을 갖는
정사각형 모양의 색종이 여러 개로 나누고 싶음



[예제] 색종이

그러나 그냥 자르지 않고, 정해진 방법을 사용해서 종이를 잘라야 함

먼저, 전체 색종이에 두 가지 색이 모두 포함되었는지 확인

1	0	1	1
0	1	1	1
0	0	1	1
0	1	1	1



[예제] 색종이

1	0	1	1
0	1	1	1
0	0	1	1
0	1	1	1

이 경우, 4등분으로 종이를 나눔

4등분으로 나눈 후, 각 조각에 대해

두 색이 모두 포함되었는지 한 번 더 확인



[예제] 색종이

1	0	1	1
0	1	1	1
0	0	1	1
0	1	1	1

왼쪽 위 조각부터 살펴보면,

0과 1을 모두 포함하고 있으니 한 번 더 나뉘어야 함

1 * 1 크기의 색종이로 자르면

결국 각각은 반드시 1개의 색만 갖게 됨

따라서 4개의 작은 색종이를 얻을 수 있음



[예제] 색종이

1	0	1	1
0	1	1	1
0	0	1	1
0	1	1	1

오른쪽 위 조각을 살펴보면?

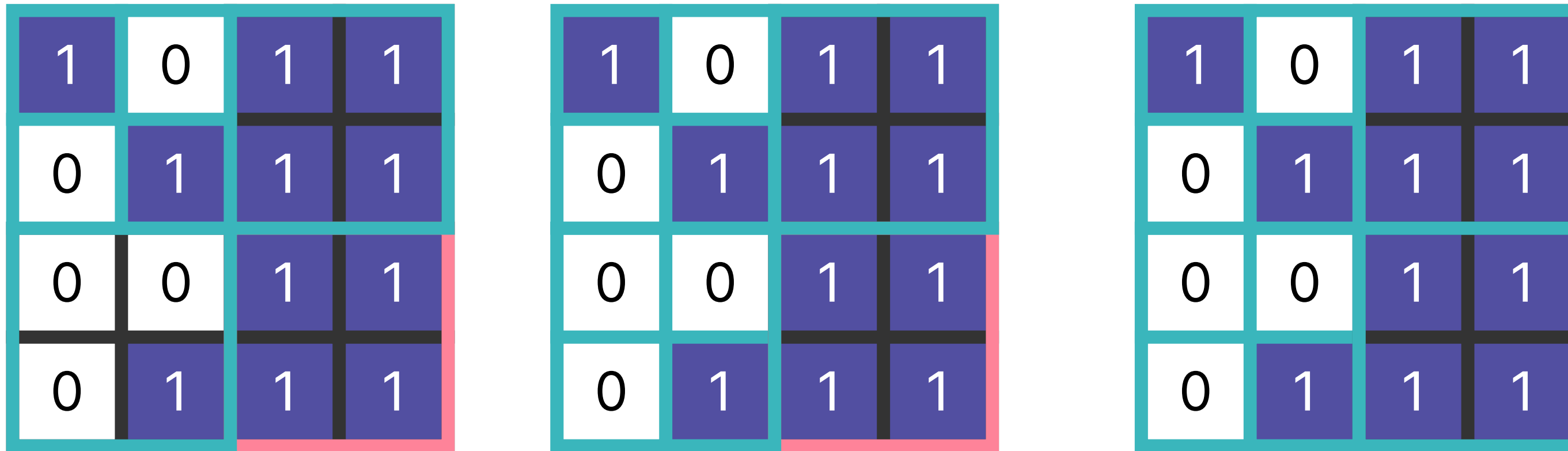
내부에 포함된 색이 모두 1,
따라서 오른쪽 윗부분은 더 이상 쪼개지 않음

즉, 1개의 색종이를 얻을 수 있음



[예제] 색종이

이런 식으로 계속 분할하며 문제를 해결해 나감



결과적으로, 1을 갖고 있는 색종이를 5개 얻을 수 있으며,

0을 갖고 있는 색종이도 5개를 얻을 수 있음



[예제] 색종이

```
def find():  
    if (...): return // 색종이의 색이 전부 같다면, 굳이 나눌 필요가  
    없음  
    find() // 왼쪽 위를 확인하기 위해 함수를 재귀적으로 실행  
    find() // 왼쪽 아래를 확인하기 위해 함수를 재귀적으로 실행  
    find() // 오른쪽 위를 확인하기 위해 함수를 재귀적으로 실행  
    find() // 오른쪽 아래를 확인하기 위해 함수를 재귀적으로 실행
```

분할 정복으로 문제를 해결하기 위해선

문제를 쪼개는 과정을 반드시 필요로 하므로,

재귀 함수가 많이 사용됨